MATLAB FINAL PROJECT

Due: Tuesday, May 05, 2026, 11:59 pm

Your objective for the final project is to develop a tool that will allow you to simulate your own three-dimensional, gravity-driven solar system. Although there is only one simple equation governing all these interactions, successfully coding this simulation will require some careful thinking and consideration of everything we've done this semester. Review the lecture slides, and check this document frequently for any updates. A few notes and guidelines on the project:

1. Data Structures.

Before deciding how to move your planets around, determine exactly how you'll store all the data required for your method. All the objects in your solar system should have at least the following properties:

- Mass
- Radius
- x, y, and z force
- x, y, and z velocity
- x, y, and z position

Since this same set of properties will be associated with every object, a structured variable array should be used to keep things organized. A suggested format is shown in the lecture slides. You're free to modify fields and names as you see fit, but you must incorporate a structured variable array into your solution.

2. Gravitational Forces.

In a simulation governing interactions between N bodies at a time, we can write the general form of the force on a generic `object(i)` as.

$$F_i = \sum_{j=1}^{N} G\frac{m_i \cdot m_j}{r^2} \qquad Note: G \approx 6.674 \times 10^{-11} N \cdot m^2/kg^2$$

where $i \neq j$, which takes into account the force contributed to `object(i)` from every other object in your system. These forces should be decomposed into an x, y, and z direction and stored in the appropriate location in your data structure. Review the lecture slides on how to decompose a 3D vector.

3. Kinematic Updates.

The forces must now be used to update the motion of each body in your system. Beginning with the equations of motion presented in lecture, use the semi-implicit Euler method to develop a discretized form, and solve for the velocities and positions at the next timestep using an appropriate step size. Run your simulation long enough to observe the desired behavior.

4. Visualizing Results.

A significant portion of your final project grade rests in how you present your results, so spend some time making your visualizations clear! You must create movie files to submit with your MATLAB code. You should submit at least the following results to showcase your method:

- A partial solar system of Sun, Earth and Moon.
- An 8-planet solar system of Sun, Mercury, Venus, Earth, Mars, Jupiter, Saturn, Uranus, Neptune.

Assign correct masses and initial velocities to create orbits. The orbits do not have to be coplanar, unidirectional, or circular.

Spend some time thinking about the following questions with your simulation tool: (i) Does the Sun in your solar system exhibit movement, and which planet(s) affect it most? (ii) What effect does an object with negative mass produce? (iii) What effect does the timestep size have on your simulation tool?

5. **Extra Credit #1 (optional).**

There's lots more you can do with your new simulation tool - binary systems, and even triple-star systems are all possible! So far, we have tried building the system where a big Sun is dominating the gradational system. In this extension, you'll develop the system(s) where there are two/three stars rotating in circular orbits in a stable manner. For simplicity of this extra credit part, you may use stars with the uniform mass in each system.

6. **Extra Credit #2 (optional).**

You may also submit a short 150-200 word reflection on how your group use MATLAB Copilot while developing your solar system simulation. Please briefly discuss:

- What things Copilot help you with, such as code structure, debugging, functions, visualization, or tests, etc.
- One useful Copilot suggestion and how you check that it is correct.
- One incomplete, incorrect, or unclear suggestion and how you revise or reject it.

Be specific and explain how you evaluate Copilot's output rather than describing your full Copilot history. Submit this reflection as a separate PDF document inside your final project zip file named "**`FP_reflection.pdf`**".

You submission of this final project should be a zip file containing all the MATLAB scripts and functions necessary for generating the video(s), as well as the video file(s) exported from MATLAB. Please use the naming convention (**`FP_MATLAB_NAME1_NAME2_NAME3.zip`**, where **`NAME1`**, **`NAME2`**, **`NAME3`** are the LastNameFirstInitial of your team members), only one team member needs to submit the zip file to Canvas Assignments. Remember to use good coding practices by keeping your code organized, choosing suitable variable names, and commenting where applicable. Any of your MATLAB .m files should contain a few comment lines at the top to provide the name of the script, a brief description of the function of the script.